

UNIVERSIDADE FEDERAL DE SERGIPE
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

CLARIANE MARIA OLIVEIRA LISBOA
IRIS AZENATE FERREIRA
JOSÉ NILTON DOS SANTOS OLIVEIRA
LEONARDO SOUZA SILVA
THIAGO DE JESUS DOS SANTOS
VICTÓRIA CARDOSO DOS SANTOS

**RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE
MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E
RETRIEVAL-AUGMENTED GENERATION (RAG).**

São Cristóvão, Sergipe
2026

CLARIANE MARIA OLIVEIRA LISBOA
ÍRIS AZENATE FERREIRA
JOSÉ NILTON DOS SANTOS OLIVEIRA
LEONARDO SOUZA SILVA
THIAGO DE JESUS DOS SANTOS
VICTÓRIA CARDOSO DOS SANTOS

**RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE
MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E
RETRIEVAL-AUGMENTED GENERATION (RAG).**

Trabalho apresentado à disciplina de Projeto e Análise
de Algoritmos da Universidade Federal de Sergipe,
como requisito parcial para obtenção de nota na
referida disciplina.

Professor: Prof. Glauco Carneiro

São Cristóvão, Sergipe
2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 METODOLOGIA.....	5
2.1 Documentação Técnica e Requisitos Exigidos.....	5
2.1.1. Fonte, Acesso e Licenciamento.....	5
2.1.2. Ambiente de desenvolvimento.....	5
2.2. Tratamentos dos textos do corpus.....	5
2.2.1 Critérios de Inclusão e Escala do Experimento.....	5
2.2.2. Limpeza, Formato e Preservação de Campos.....	6
2.2.3. Limitações de Cobertura e Análise de Viés.....	6
2.2.4. Justificativa da Estratégia de Chunking.....	6
2.3. Definição formal do problema.....	7
2.4. Definição de Relevância.....	9
2.5. Métodos de Busca e Ordenação.....	10
2.5.1 Busca Linear.....	10
2.5.2 Merge Sort.....	11
2.5.2.1 Estrutura do algoritmo e pseudocódigo.....	11
2.5.3 Corretude por indução forte (merge_sort).....	13
2.5.4. Hipótese de indução.....	13
2.5.5 Caso base ou caso trivial.....	13
2.5.6 Passo indutivo ($N > 1$).....	13
2.5.6.1 Corretude por invariante de laço (merge).....	14
2.5.7 Enunciado.....	14
2.6 Índice Invertido.....	15
2.6.1 Por que cada termo é inserido uma vez por documento (uso de set).....	16
2.6.2 Pseudocódigo da construção e da consulta.....	17
2.7 Análise Assintótica.....	18
2.7.1 Análise da busca linear.....	19
2.7.2 Análise da busca indexada.....	22
2.7.3 Análise do Merge Sort.....	23
2.7.4 Síntese comparativa e fatores fora do modelo RAM.....	25
2.8 Precision@5.....	26
2.9 Desenho Experimental.....	26
3. RESULTADOS.....	28
4 DISCUSSÃO.....	32
4.1 Escalabilidade, custos e trade-offs.....	32
4.2 Relação com a recuperação de contexto em aplicações de IA generativa (RAG).....	33
4.3 Limitações e ameaças à validade do experimento.....	34
5 DECLARAÇÃO DE USO DE IA GENERATIVA.....	36
6 TABELA DE CONTRIBUIÇÃO INDIVIDUAL.....	37
7 URL DO VÍDEO E REPOSITÓRIO.....	38
REFERÊNCIAS BIBLIOGRÁFICAS.....	39

1 INTRODUÇÃO

A recuperação de informações em coleções de documentos envolve a identificação de conteúdos relacionados a uma determinada consulta. Em coleções de artigos científicos, esse processo permite localizar publicações relevantes sobre um tema, facilitando o acesso às informações disponíveis. Entretanto, à medida que a quantidade de documentos aumenta, a escolha da estratégia de busca e ordenação pode influenciar o tempo necessário para recuperar os resultados e os recursos computacionais utilizados (Manning; Raghavan; Schütze, 2008).

Esse processo também está relacionado a aplicações de inteligência artificial generativa que utilizam Retrieval-Augmented Generation (RAG). Nesses sistemas, documentos relevantes são recuperados e fornecidos como contexto a um modelo de linguagem, que utiliza essas informações durante a geração de respostas (Lewis et al., 2020). Dessa forma, a eficiência da recuperação e a relevância dos documentos selecionados são aspectos importantes para o funcionamento dessas aplicações.

Existem diferentes estratégias para realizar a recuperação de informações. A Busca Linear percorre todos os documentos de uma coleção para identificar aqueles relacionados à consulta. O Índice Invertido, por sua vez, permite localizar diretamente os documentos associados aos termos pesquisados, reduzindo a necessidade de examinar toda a coleção (Manning; Raghavan; Schütze, 2008). Além da busca, a ordenação dos resultados é uma etapa importante, pois permite apresentar os documentos de acordo com suas pontuações de relevância.

Diante dessas diferentes abordagens, torna-se importante compreender como as estratégias de recuperação e ordenação se comportam em relação ao custo computacional e à qualidade dos resultados recuperados. A comparação entre elas permite analisar as diferenças no processamento das consultas e os efeitos do aumento da quantidade de documentos. Essa análise também se relaciona aos desafios de eficiência dos sistemas RAG, que envolvem o equilíbrio entre a qualidade da recuperação e os custos computacionais (Gan et al., 2025).

Nesse contexto, o presente trabalho tem como objetivo comparar experimentalmente três configurações de recuperação e ordenação de informações: Busca Linear, Índice Invertido e Busca Linear combinada com o algoritmo Merge Sort. Para isso, utiliza-se uma coleção de artigos científicos de acesso aberto da ACL Anthology, relacionados a modelos de linguagem, recuperação de informação e RAG. A comparação considera a eficiência computacional e a qualidade da recuperação, buscando analisar o comportamento das configurações diante de diferentes tamanhos de corpus.

2 METODOLOGIA

2.1 Documentação Técnica e Requisitos Exigidos

2.1.1. Fonte, Acesso e Licenciamento

Os dados utilizados neste trabalho foram obtidos no repositório oficial da ACL Anthology (<https://aclanthology.org/>), que reúne publicações científicas da área de Processamento de Linguagem Natural e Linguística Computacional. A coleta foi realizada de forma automatizada por meio da biblioteca Python *acl-anthology*, utilizando o método *Anthology.from_repo()*. Essa estratégia possibilitou a extração direta dos dados disponibilizados pelo repositório, sem a utilização de fontes secundárias.

A ACL Anthology disponibiliza seu acervo sob a licença *Creative Commons Attribution 4.0 International* (CC BY 4.0), que permite o uso e a redistribuição do material, desde que seja atribuída a autoria e indicada a fonte original. Os dados foram utilizados exclusivamente para fins acadêmicos, e o acesso ao repositório ocorreu em setembro de 2026.

2.1.2. Ambiente de desenvolvimento

A coleta, o tratamento e a recuperação dos documentos foram implementados na linguagem Python, em um notebook executado no ambiente Jupyter Notebook. Esse formato permitiu organizar o experimento em etapas, registrar os parâmetros empregados e reproduzir as execuções realizadas. Também foram utilizadas bibliotecas específicas para manipulação de dados, processamento textual, recuperação dos artigos e medição do desempenho dos algoritmos.

O ambiente computacional utilizado foi composto por Python na versão 3.13.15, Jupyter Notebook na versão 6.5.7 e pelas bibliotecas *acl-anthology* 1.3.3 *pandas* 2.2.3, *NumPy* 2.1.3 e *sentence-transformers* 5.7.0. As execuções foram realizadas no Google Colab, utilizando o sistema operacional Linux-6.6.122+-x86_64-with-glibc2.39.

2.2. Tratamentos dos textos do corpus

2.2.1 Critérios de Inclusão e Escala do Experimento

Foram incluídos no corpus somente os artigos que apresentavam os campos *title* e *abstract* preenchidos. A coleta inicial considerou até 18.000 artigos candidatos da ACL Anthology. Para identificar documentos relacionados ao tema do experimento, foi utilizada uma consulta composta pelos termos *retrieval augmented generation*, *information retrieval* e *large language model*.

Após a aplicação da pontuação lexical, foram mantidos os documentos que obtiveram pontuação maior que zero ($_score > 0$), isto é, aqueles que continham pelo menos um dos termos considerados na consulta. Em seguida, os documentos foram ordenados pela pontuação lexical, e os primeiros resultados foram selecionados para formar dois recortes experimentais, contendo $N = 500$ e $N = 1000$ documentos. Esses recortes foram utilizados na matriz experimental composta por 1800 execuções mensuráveis.

2.2.2. Limpeza, Formato e Preservação de Campos

O processamento textual foi realizado a partir da concatenação dos campos *title* e *abstract*. Inicialmente, os textos foram convertidos para letras minúsculas. Em seguida, foram removidos os caracteres especiais, preservando-se apenas caracteres alfanuméricos e espaços por meio da expressão regular $[\^a-z0-9\s]$. Por fim, os múltiplos espaços em branco foram substituídos por um único espaço.

Para preservar a identificação e a rastreabilidade das publicações, foram mantidos os campos *id*, correspondente ao *full_id* da ACL Anthology, *title*, *abstract*, *year*, *doi* e *url*.

2.2.3. Limitações de Cobertura e Análise de Viés

A cobertura do corpus está limitada às publicações disponíveis na ACL Anthology, incluindo trabalhos provenientes de conferências como ACL, EMNLP e NAACL. Dessa maneira, o conjunto de dados não contempla publicações indexadas exclusivamente em outras fontes.

Essa delimitação pode introduzir um viés de cobertura, pois concentra a amostra na comunidade de Processamento de Linguagem Natural e Linguística Computacional. Consequentemente, os resultados do experimento não devem ser generalizados diretamente para todas as áreas nas quais técnicas de recuperação de informação e RAG são utilizadas.

2.2.4. Justificativa da Estratégia de Chunking

Neste trabalho, cada artigo foi representado por um único *chunk*, formado pela união de seu título e resumo. Essa estratégia foi adotada porque esses campos sintetizam o objetivo, a metodologia e os principais resultados da publicação, evitando informações secundárias presentes no texto completo.

A utilização de um artigo como uma única unidade de recuperação também reduziu a necessidade de baixar e processar arquivos PDF completos, diminuindo o tempo de

preparação e o consumo de memória durante a indexação e a recuperação. Além disso, essa configuração preservou o título e o resumo como uma unidade textual única e coerente. Entretanto, como limitação, a estratégia pode deixar de recuperar informações específicas presentes somente no corpo do artigo.

2.3. Definição formal do problema

O problema abordado neste trabalho consiste em recuperar os artigos científicos mais relacionados a uma consulta. A entrada é formada pelo corpus C , pela consulta q e pelo parâmetro inteiro k , definido como $k = 5$ no experimento. Esse parâmetro determina a quantidade máxima de documentos retornados. Cada documento $d_i \in C$ é representado por um identificador, pelo texto resultante da concatenação de seu título e resumo e pela lista de tokens obtida após o tratamento textual.

A saída é uma lista R , contendo até k documentos pertencentes ao corpus. Os documentos são selecionados e ordenados de acordo com uma pontuação lexical, calculada pela soma das ocorrências dos termos da consulta no texto de cada documento. Formalmente, a pontuação é definida por:

$$score(d_i, q) = \sum_{t \in T(q)} freq(t, d_i)$$

Nessa expressão, $T(q)$ representa a sequência de termos obtida após a normalização, tokenização, aplicação de *stemming* e remoção de *stopwords* da consulta. As repetições são preservadas, de modo que um termo repetido na consulta contribui mais de uma vez para a pontuação. Por sua vez, $freq(t, d_i)$ representa a quantidade de ocorrências do termo t no documento d_i .

Os documentos recuperados são ordenados de forma não crescente de pontuação. Não foi estabelecido um critério adicional de desempate comum às três configurações. Dessa forma, a ordem entre documentos com a mesma pontuação decorre do comportamento da estrutura de dados e do algoritmo de ordenação utilizados em cada configuração.

Para a realização do experimento, foram definidas dez consultas em inglês, identificadas de Q1 a Q10, relacionadas à recuperação de informação, aos modelos de linguagem e à técnica Retrieval-Augmented Generation (RAG). O Quadro 1 apresenta as consultas utilizadas nas três configurações avaliadas.

Quadro 1 - Consultas utilizadas na avaliação experimental.

Identificador	Definição
Q1	<i>What information retrieval methods are used in RAG systems?</i>
Q2	<i>Which studies compare sparse retrieval and dense retrieval?</i>
Q3	<i>What semantic search techniques are used to retrieve context for language models?</i>
Q4	<i>What reranking methods are used in retrieval systems for LLMs?</i>
Q5	<i>What metrics are used to evaluate Retrieval-Augmented Generation systems?</i>
Q6	<i>Which papers use embeddings for document retrieval?</i>
Q7	<i>What techniques are used to select the most relevant documents or excerpts for a language model?</i>
Q8	<i>Which methods were compared for context retrieval?</i>
Q9	<i>What limitations of RAG systems are reported in the articles?</i>
Q10	<i>What chunking strategies are used in document retrieval?</i>

Fonte: Elaborado pelos autores.

Como pré-condições, considera-se que o corpus C foi previamente preparado e que cada documento possui um identificador, um texto e uma lista de tokens. Considera-se também que a consulta q pode ser processada pela mesma representação textual adotada no corpus e que k é um número inteiro. Os procedimentos empregados na preparação dos documentos e das consultas são apresentados na Seção 2.2.

Ao término da execução, a lista R deve satisfazer as seguintes pós-condições:

1. conter no máximo k documentos quando $k > 0$;
2. conter somente documentos pertencentes ao corpus C ;
3. não apresentar documentos duplicados;
4. estar ordenada de forma decrescente de pontuação;

Se o corpus estiver vazio, as três configurações retornam uma lista vazia. Quando k é igual a zero, também é retornada uma lista vazia. Para consultas vazias, formadas somente por *stopwords* ou compostas apenas por termos inexistentes no corpus, a Busca Linear e a

configuração com Merge Sort atribuem pontuação zero aos documentos. Nessas mesmas situações, o Índice Invertido retorna uma lista vazia, pois nenhum documento candidato é identificado.

Para verificar essas condições, foram implementados testes automatizados envolvendo corpus vazio, consulta vazia, consulta formada apenas por stopwords, termos inexistentes, valor não positivo de k e equivalência dos documentos retornados pelas três configurações. Também foram realizados testes específicos de corretude da ordenação produzida pelo Merge Sort, comparando seus resultados com a ordenação de referência do Python.

2.4. Definição de Relevância

Para avaliar a qualidade dos resultados recuperados, foi construído um gabarito semântico independente dos algoritmos de busca avaliados. Essa separação foi necessária porque os três algoritmos utilizam uma pontuação lexical baseada na frequência dos termos da consulta. Caso a relevância também fosse definida somente pela presença desses termos, a avaliação utilizaria um critério semelhante ao empregado pelos próprios algoritmos, o que poderia favorecer artificialmente seus resultados.

O gabarito foi construído com o modelo de embeddings *all-MiniLM-L6-v2*, disponibilizado pelo framework *Sentence Transformers* (Sentence Transformers, 2021?). Para cada consulta e para cada documento do corpus, considerando a concatenação de seu título e resumo, o modelo gerou uma representação vetorial. Em seguida, foi calculada a similaridade de cosseno entre o vetor da consulta e o vetor de cada documento. A utilização de embeddings permite comparar os textos com base em sua proximidade semântica, mesmo quando eles não apresentam exatamente as mesmas palavras (Reimers; Gurevych, 2019).

Para cada consulta, os documentos foram ordenados de forma decrescente conforme a similaridade de cosseno. Os 15 documentos com os maiores valores de similaridade formaram o gabarito semântico e foram considerados relevantes para aquela consulta.

O gabarito foi utilizado exclusivamente no cálculo da Precision@5 e não participou da pontuação, da seleção dos candidatos ou da ordenação realizada pela Busca Linear, pelo Índice Invertido ou pelo Merge Sort. Portanto, os algoritmos continuaram realizando uma recuperação lexical, enquanto a avaliação verificou em que medida os documentos recuperados correspondiam a uma referência semântica comum às três configurações.

2.5. Métodos de Busca e Ordenação

Neste trabalho, foram avaliadas três configurações. A primeira, denominada Busca Linear, percorre todos os documentos do corpus, calcula suas pontuações e utiliza o método `sort()` do Python para ordenar os resultados. A segunda utiliza um Índice Invertido para selecionar apenas os documentos candidatos associados aos termos da consulta, também empregando o `sort()` do Python na ordenação. A terceira, denominada Merge Sort, utiliza a mesma varredura completa do corpus e a mesma função de pontuação da Busca Linear, diferenciando-se exclusivamente pela utilização do Merge Sort implementado na etapa de ordenação.

2.5.1 Busca Linear

A Busca Linear foi utilizada como uma das configurações do experimento e consistiu na avaliação sequencial de todos os documentos do corpus para uma determinada consulta. Para cada documento, foi calculada uma pontuação (*score*) com base na frequência dos termos da consulta presentes no texto do documento.

Inicialmente, a consulta foi normalizada e tokenizada, com a remoção dos termos definidos como *stopwords*. Em seguida, cada documento foi percorrido individualmente e sua pontuação foi calculada pela função `score_documento()`. A pontuação correspondeu à soma das ocorrências dos termos da consulta no texto do documento.

Durante esse processo, foi contabilizada uma comparação para cada documento avaliado. Como a Busca Linear percorreu todos os documentos do corpus, o número de comparações correspondeu ao tamanho do corpus utilizado na execução.

Após o cálculo das pontuações, os documentos foram organizados em ordem decrescente de *score* utilizando a função `sort()` da linguagem Python. Por fim, foram selecionados os cinco primeiros documentos da lista ordenada para compor os resultados da consulta. O Algoritmo 1 apresenta o pseudocódigo desta configuração.

Algoritmo 1 - Busca linear.

```
função busca_linear(documentos, consulta, k)
    resultados = lista vazia
    comparações = 0

    para cada documento em documentos:
        score = PONTUAÇÃO(documento, consulta)
```

```

    comparações = comparações + 1
    adicione (score, documento) a resultados

ordene resultados em ordem decrescente de score
retorne os k primeiros elementos de resultados, comparações

```

Fonte: Elaborado pelos autores.

A Busca Linear foi utilizada como referência para a comparação com as demais configurações, uma vez que todos os documentos do corpus foram avaliados durante cada consulta.

2.5.2 Merge Sort

Para a ordenação dos dados, utilizou-se o algoritmo *Merge Sort* configurado em ordem decrescente. O estudo fundamenta-se em uma demonstração matemática formal e em uma análise rigorosa de complexidade, estruturada sob a seguinte metodologia:

2.5.2.1 Estrutura do algoritmo e pseudocódigo

Para fundamentar a análise formal, considera-se a implementação em pseudocódigo do algoritmo Merge Sort padrão que opera sobre uma lista de entrada contendo elementos comparáveis. Seguindo a lógica do algoritmo, o objetivo é produzir uma permutação ordenada da lista original tal que a ordem decrescente seja garantida.

Algoritmo 2 - Merge Sort.

```

função merge_sort(lista)
    se tamanho(lista) <= 1
        retorne lista

    meio = tamanho(lista) / 2
    esquerda = merge_sort(lista[0 até meio])
    direita = merge_sort(lista[meio até o fim])

    retorne merge(esquerda, direita)

função merge(esquerda, direita)
    resultado = lista vazia
    enquanto esquerda e direita tiverem elementos:
        se esquerda[0] >= direita[0]:

```

```

    adicione esquerda[0] ao resultado
    remova o elemento da esquerda
senão:
    adicione direita[0] ao resultado
    remova o elemento da direita

adicione o restante da esquerda e da direita ao resultado
retorne resultado

```

Fonte: Elaborado pelos autores.

Como é possível perceber o algoritmo pode ser dividido em duas partes, a função *merge_sort* onde dividimos a lista na metade até restar apenas um elemento, dessa forma, como só há um elemento, ele está forçadamente ordenado com ele mesmo. E a função *merge* que é justamente a combinação linear desses vetores decrescente forçadamente ordenados em uma lista única ordenada de forma decrescente. O *merge_sort* exige demonstração por indução forte, uma vez que ele possui recursão e o *merge* por invariante de laço. Abaixo segue um quadro comparativo das partes do código.

Quadro 2 - Comparação das funções *merge_sort* e *merge*.

Componente	Papel Funcional	Método de Demonstração
<i>merge_sort</i>	Divisão recursiva do vetor em subvetores de tamanho aproximado $N/2$ até o caso base.	Indução Matemática Forte / Estrutural
<i>merge</i>	Combinação linear de dois vetores já ordenados de forma decrescente em uma lista única unificada.	Invariante de Laço (Inicialização, Manutenção e Término)

Fonte: Elaborado pelos autores.

2.5.3 Corretude por indução forte (merge_sort)

Refletindo a divisão estrutural do algoritmo em duas etapas, a demonstração de corretude foi equivalentemente modularizada. Dada a natureza recursiva da primeira fase, sua validação formal foi estabelecida por meio do princípio da indução matemática forte.

2.5.4. Hipótese de indução

Para provarmos qualquer suposição precisamos inicialmente de uma base sólida e verdadeira, dessa forma, assume-se que a função `merge_sort` executa com sucesso e ordena corretamente qualquer lista de cardinalidade k , onde $k < N$, retornando uma permutação válida e estritamente ordenada em ordem decrescente.

2.5.5 Caso base ou caso trivial

Provamos a hipótese de indução no caso mais simples possível, chamamos esse caso de caso trivial.

Se a lista contém zero ou um elemento ($N = 0$ ou $N = 1$):

- Uma lista vazia ou unielementar não possui pares de elementos (x, y) fora de ordem, estando portanto trivialmente ordenada.
- O algoritmo avalia a condição de guarda e retorna a lista imediatamente sem modificações.
- A saída preserva o multiconjunto de elementos original e satisfaz a propriedade de ordenação. O caso base é, portanto, válido

2.5.6 Passo indutivo ($N > 1$)

Considere uma lista de entrada N com cardinalidade arbitrária $N > 1$. O particionamento estrutural do algoritmo divide N em duas sublistas:

- Partição Esquerda: de tamanho aproximado $N/2$. Como $N > 1$, seu tamanho é menor que N . Chamaremos de PE
- Partição Direita: de tamanho aproximado $N/2$. Como $N > 1$, seu tamanho é menor que N . Chamaremos de PD

Dado que $N > 1$, decorre algebricamente que $1 \leq PE < N$ e $1 \leq PD < N$. Pelo fato de a cardinalidade de ambas as partições ser estritamente menor que N e maior ou igual ao caso base, seguindo a hipótese de indução temos que PE e PD são válidas e retornam uma

permutação válida e estritamente ordenada em ordem decrescente.

Pela hipótese de indução aplicável diretamente a ambas as chamadas recursivas:

- `merge_sort(esquerda)` retorna uma sublista perfeitamente ordenada em ordem decrescente, contendo exatamente os elementos da partição esquerda.
- `merge_sort(direita)` retorna uma sublista perfeitamente ordenada em ordem decrescente, contendo exatamente os elementos da partição direita.

2.5.6.1 Corretude por invariante de laço (merge)

Complementando a prova recursiva, a demonstração de corretude por ser um processo iterativo apoia-se na formulação de um invariante de laço para a função *merge*. O papel dessa subrotina é intercalar duas sequências decrescentes, consolidando-as na lista final (resultado). A prova matemática dessa união é desenvolvida a partir do enunciado do invariante, seguido pela validação rigorosa de seus três estágios fundamentais: inicialização, manutenção e término do laço

2.5.7 Enunciado

De antemão é possível notar que no início de cada iteração do laço temos:

1. **Ordenação:** A lista “resultado” contém os maiores elementos processados até o momento, dispostos em ordem decrescente.
2. **Majorização:** Qualquer elemento já presente em resultado é maior ou igual aos elementos de `esquerda[0]` e `direita[0]`.
3. **Conservação:** A união de resultado, esquerda e direita forma os mesmos elementos que entraram na chamada da função.

Inicialização

Antes da primeira iteração, a lista resultado está vazia:

- A condição de ordenação é vacuamente verdadeira.
- As listas originais mantêm-se inalteradas, logo não há perda de elementos. O invariante se sustenta.

Manutenção

A cada passo, comparamos as frentes `esquerda[0]` e `direita[0]`:

- Sendo esquerda e direita ordenadas de forma decrescente, o maior valor geral só pode ser uma de suas cabeças.
- Ao extrair o maior deles (usando a condição $\text{esquerda}[0] \geq \text{direita}[0]$) e movê-lo para o fim de resultado, nós expandimos o vetor mantendo a ordem não-crescente.

Término

O laço encerra quando uma lista esvazia. Como a sublista restante ainda está perfeitamente ordenada e seus elementos são menores ou iguais ao último de resultado, anexá-los ao fim completa a permutação ordenada da lista unida.

Abaixo segue um quadro explicativo de cada uma das etapas do invariante de laço.

Quadro 3 - Etapas da prova invariante de laço.

Fase do Invariante	Garantia Matemática
Inicialização	Condição de ordenação é vacuamente verdadeira. Nenhum elemento perdido.
Manutenção	O maior elemento não processado é adicionado, mantendo a monotonicidade decrescente.
Término	Elementos restantes são menores ou iguais ao último e já estão ordenados; adicioná-los encerra a prova.

Fonte: Elaborado pelos autores.

Por fim, a união dos dois passos formais, indução forte e invariante de laço, completa a demonstração rigorosa de corretude do Merge Sort. A função intercala monotonicamente listas e as divide sucessivamente para estruturar uma ordenação decrescente perfeita, mantendo a conservação dos elementos.

2.6 Índice Invertido

O índice invertido é uma estrutura de dados do tipo dicionário (*hash map*), na qual cada chave é um termo do vocabulário e cada valor é uma lista de *postings*, os índices (ou identificadores) dos documentos em que aquele termo ocorre pelo menos uma vez.

Formalmente:

$$\text{indice}: T \rightarrow \mathcal{P}(\{0, \dots, N - 1\})$$

onde T é o vocabulário (conjunto de termos distintos do corpus, após normalização e stemming) e $\text{índice}(t)$ é o conjunto de documentos que contêm o termo t .

Exemplo ilustrativo, considerando três documentos:

- *D0: "retrieval augmented generation for language models"*
- *D1: "dense retrieval and language representation"*
- *D2: "large language model pretraining"*

Após tokenização, o índice resultante seria:

retrieval	> [0, 1]
augmented	-> [0]
generation	-> [0]
language	-> [0, 1, 2]
model	-> [0, 2]
dense	-> [1]
representation	-> [1]
large	-> [2]
pretraining	-> [2]

Pode-se notar que *language* aparece nos três documentos, e por isso sua lista de *postings* tem tamanho 3; já *dense* é exclusivo de D1. É essa concentração desigual, pois alguns termos raros com listas curtas, poucos termos frequentes com listas longas, o que permite ao índice invertido reduzir o espaço de busca na maioria das consultas.

2.6.1 Por que cada termo é inserido uma vez por documento (uso de *set*)

Na implementação, a etapa de indexação por documento é feita da seguinte forma:

```
palavras = set(tokenizar(doc["text"]))
```

O uso de *set()* converte a lista de tokens (que pode conter repetições) em um conjunto de termos únicos antes de povoar o índice. Isso é necessário por duas razões:

1. Semântica do índice invertido. A pergunta que o índice responde é "quais documentos contêm o termo t ?" É uma pergunta binária de pertencimento, não de contagem. A lista de *postings* de t deve conter o índice de d no máximo uma vez, independentemente de quantas vezes t ocorra dentro de d .
2. Evitar redundância e inflar custo. Sem o *set*, um documento com o termo *language* repetido 10 vezes inseriria i dez vezes na mesma lista de *postings*, distorcendo o tamanho de `indice[t]` sem agregar nenhuma informação nova, o que aumentaria artificialmente $O(L)$ na construção e $O(|\text{candidatos}|)$ na consulta, já que *candidatos* seria populado com duplicatas do mesmo i .

A frequência do termo dentro do documento não é descartada: ela continua disponível em `documento["tokens"]` e é recalculada, quando necessário, por `score_documento`. O índice invertido armazena apenas presença; a frequência é responsabilidade da função de relevância, aplicada somente sobre os candidatos.

2.6.2 Pseudocódigo da construção e da consulta

Algoritmo 3 - Índice invertido.

```

função construir_indice_invertido(documentos)
    indice = dicionário vazio
    para i de 0 até N-1:
        palavras = conjunto_unico(tokenizar(documentos[i].text))
        para cada palavra em palavras:
            se palavra não está em indice:
                indice[palavra] = lista vazia
            adicione i à indice[palavra]
    retorne indice

função busca_indexada(documentos, indice, consulta, k)

    candidatos = conjunto vazio

    para cada termo em tokens_da_consulta(consulta):
        se termo está em indice:
            candidatos = candidatos U indice[termo]

    resultados = lista vazia

    para cada i em candidatos:

```

```

score = score_documento(documentos[i], consulta)

adicione (score, documentos[i]) a resultados

ordene resultados por score em ordem decrescente

retorne resultados[0 até k]

```

Fonte: Elaborado pelos autores.

2.7 Análise Assintótica

A análise assintótica foi realizada com base no modelo RAM (Random Access Machine), no qual operações elementares, como acesso a uma posição de uma lista, comparação, atribuição e operações aritméticas simples, são consideradas de custo constante. Esse modelo permite descrever o crescimento do tempo e do espaço utilizados pelos algoritmos em função do tamanho da entrada, independentemente da tecnologia ou da máquina específica em que o algoritmo é executado (Erickson, 2019).

Para uniformizar a análise dos algoritmos, foram definidos os parâmetros apresentados no Quadro 4.

Quadro 4 - Parâmetros utilizados na análise assintótica.

Parâmetro	Definição
N	Número de documentos ou chunks presentes no corpus.
L	Número total de tokens presentes nos documentos do corpus.
m	Número de termos da consulta após a remoção de <i>stopwords</i> e a normalização.
r	Número de documentos candidatos recuperados pelo índice invertido.
k	Número de resultados retornados ao final da busca.
d	Dimensão dos <i>embeddings</i> .

Fonte: Elaborado pelos autores.

O parâmetro d não se aplica diretamente aos três algoritmos de recuperação analisados, pois eles utilizam representação lexical. No código, os embeddings do modelo *all-MiniLM-L6-v2* foram empregados apenas em uma etapa auxiliar para construir o gabarito

semântico. Este gabarito corresponde ao conjunto de documentos considerados relevantes para cada consulta com base na similaridade semântica e foi utilizado como referência no cálculo da Precision@5 , sem participação nos algoritmos de busca linear, indexada ou no Merge Sort.

2.7.1 Análise da busca linear

Conforme apresentado no pseudocódigo do Algoritmo 1, a busca linear percorre todos os N documentos do corpus. Para cada documento, o algoritmo calcula uma pontuação lexical por meio da soma das ocorrências dos termos da consulta. Em seguida, os resultados são armazenados, ordenados de forma decrescente pela pontuação e limitados aos k primeiros elementos.

As principais operações elementares desta implementação são o acesso aos documentos e aos seus tokens, a comparação dos termos, a contagem de ocorrências, a inserção dos resultados na lista e as comparações realizadas durante a ordenação.

Uma análise simplificada, semelhante ao modelo clássico de busca linear ($O(N)$ no pior caso, considerando a comparação de cada elemento com o valor buscado), poderia levar a crer que a avaliação dos documentos tem custo $O(Nm)$. Essa representação genérica, porém, não descreve integralmente a situação analisada aqui, na qual cada documento não é apenas comparado, mas pontuado a partir da contagem de ocorrências dos m termos da consulta em seus tokens.

No pseudocódigo da busca linear, o cálculo da pontuação de cada documento é representado por uma operação auxiliar. Para detalhar o funcionamento e o custo dessa operação, o pseudocódigo do Algoritmo 4 apresenta separadamente os procedimentos PONTUAÇÃO e CONTAR.

Algoritmo 4 - Cálculo da pontuação lexical de um documento.

```
função pontuação(documento, consulta)
    score = 0
    para cada termo em consulta:
        score = score + contar(documento.tokens, termo)
    retorne score

função contar(tokens, termo)
```

```

ocorrências = 0

para cada token em tokens:
    se token = termo:
        ocorrências = ocorrências + 1

retorne ocorrências

```

Fonte: Elaborado pelos autores.

A operação CONTAR percorre todos os tokens de um documento para determinar quantas vezes um termo aparece. Considerando ℓ_i como a quantidade de tokens do documento i , uma execução dessa operação apresenta custo $\Theta(\ell_i)$. Como a contagem é realizada para cada um dos m termos da consulta, o custo da pontuação de um documento é:

$$T_i = \Theta(m\ell_i)$$

A busca linear executa esse procedimento para todos os N documentos. Portanto, o custo total da pontuação corresponde a:

$$T_{\text{pontuação}} = m \sum_{i=1}^N \ell_i$$

Como L representa a quantidade total de tokens do corpus:

$$L = \sum_{i=1}^N \ell_i$$

obtém-se:

$$T_{\text{pontuação}} = \Theta(mL)$$

Após o cálculo das pontuações, os N resultados são armazenados em uma lista e ordenados de forma decrescente. Na implementação feita, essa ordenação é realizada pelo método *sort* da linguagem Python. No caso médio e no pior caso, essa etapa apresenta complexidade $\Theta(N \log N)$. Quando os elementos já se encontram ordenados ou quase ordenados, o algoritmo de ordenação pode aproveitar essa disposição, aproximando-se de $\Theta(N)$ no melhor caso.

Após a ordenação, é realizado o fatiamento da lista para retornar somente os k primeiros resultados. A criação dessa lista possui custo $\Theta(k)$. Desse modo, o custo total da busca linear pode ser representado por:

$$T_{busca\ linear} = \Theta(mL) + \Theta(N \log N) + \Theta(k)$$

O fatiamento retorna no máximo $\min(k, N)$ elementos, pois, quando $k > N$, são retornados apenas os N documentos existentes. Como esse valor nunca excede N , o termo $\Theta(k)$ não modifica a ordem assintótica dominante. Portanto, para os casos médio e pior, a complexidade total é:

$$T_{médio} = T_{pior} = \Theta(mL + N \log N)$$

No melhor caso da ordenação, a complexidade é:

$$T_{melhor} = \Theta(mL + N)$$

A etapa de pontuação permanece $\Theta(mL)$ nos três casos, pois esta implementação não possui critério de parada antecipada: todos os termos da consulta são contados em todos os documentos, independentemente de serem encontrados ou não.

Em relação ao espaço, a busca armazena uma tupla contendo a pontuação e a referência de cada um dos N documentos, o que exige espaço $O(N)$. A lista final contém k elementos e ocupa $O(k)$. Além disso, a ordenação pode utilizar memória auxiliar proporcional a N . Como $k \leq N$, a complexidade espacial dominante da busca linear é:

$$S_{busca\ linear} = O(N)$$

Assim, embora a busca percorra linearmente os documentos, seu custo não pode ser representado apenas por $O(N)$ ou $O(Nm)$. A implementação também considera o tamanho dos documentos durante a contagem das ocorrências e realiza a ordenação de todas as pontuações. Por isso, sua complexidade temporal é expressa por $\Theta(mL + N \log N)$ nos casos médio e pior.

2.7.2 Análise da busca indexada

O índice invertido associa cada termo encontrado no corpus a uma lista de postagem, isto é, uma lista que contém os identificadores dos documentos nos quais esse termo aparece. Essa estrutura permite acessar diretamente os documentos relacionados aos termos da consulta, sem percorrer todo o corpus (Manning; Raghavan; Schütze, 2008).

Para construir o índice utilizado neste trabalho, todos os documentos são percorridos e tokenizados. Em seguida, cada termo é inserido no dicionário e associado aos identificadores dos documentos nos quais aparece.

As principais operações elementares desta implementação são o acesso e a inserção de chaves no dicionário (índice), a união de conjuntos de candidatos durante a consulta, o acesso aos tokens dos documentos candidatos durante a pontuação e as comparações realizadas durante a ordenação dos resultados.

Como todos os L tokens do corpus precisam ser processados, o tempo esperado de construção é $O(L)$. O número de associações entre termos e documentos é limitado pelo total de tokens processados. Portanto, o espaço utilizado pelo índice também pode ser representado pela mesma expressão.

Durante a consulta, o algoritmo acessa as listas de postagem correspondentes aos m termos e reúne os documentos encontrados em um conjunto de candidatos. Se P_q representar o número total de elementos percorridos nessas listas, o custo esperado para localizar os candidatos é:

$$O(m + P_q)$$

Depois dessa etapa, apenas os r documentos candidatos têm suas pontuações calculadas. Se L_r representar o total de tokens desses candidatos, o custo dessa operação é:

$$\Theta(mL_r)$$

Assim, o custo total da consulta indexada pode ser representado por:

$$O(m + P_q + mL_r + r \log r)$$

No melhor caso, nenhum dos termos da consulta aparece no índice. Nesse cenário, não há candidatos, e o custo da consulta é $O(m)$.

No caso médio, apenas parte do corpus é recuperada, de modo que $r < N$ e $L_r < L$, e o custo corresponde à expressão geral apresentada anteriormente.

No pior caso, os termos aparecem em quase todos os documentos, de modo que $r = N$ e $L_r = L$. Nesse cenário, a busca indexada precisa percorrer listas de postagem extensas, calcular a pontuação dos N documentos candidatos e ordenar os resultados. Portanto, o custo se aproxima do observado na busca linear:

$$T_{pior} = \Theta(mL + N \log N)$$

O termo $\Theta(mL)$ corresponde ao cálculo das pontuações dos documentos candidatos, enquanto $\Theta(N \log N)$ corresponde à ordenação dos resultados. Assim, no pior caso, quando o índice não consegue reduzir o conjunto de candidatos, a busca indexada não apresenta vantagem assintótica em relação à busca linear.

Durante a consulta, o conjunto de candidatos e a lista de resultados utilizam espaço $O(r)$. Além disso, o índice previamente construído ocupa espaço $O(L)$. O índice invertido exige, portanto, memória adicional e um custo inicial de construção, mas pode reduzir o tempo das consultas posteriores.

2.7.3 Análise do Merge Sort

O algoritmo *Merge Sort*, apresentado e formalmente validado na seção 2.5, divide recursivamente a lista de pares (pontuação, documento) ao meio até restarem sublistas de tamanho unitário, e a função *merge* intercala essas sublistas já ordenadas em uma única lista decrescente. Enquanto a seção 2.5 tratou da corretude dessas duas rotinas, esta seção analisa seu custo assintótico no modelo RAM.

As principais operações elementares desta implementação são a chamada recursiva sobre cada metade da lista, a comparação entre os elementos de topo das duas sublistas na função *merge*, a atribuição do elemento escolhido à lista resultado e a remoção do elemento correspondente da sublista de origem.

A rotina *merge*, responsável por combinar duas sublistas de tamanho total n , percorre cada elemento uma única vez até esvaziar ambas as entradas, o que lhe confere custo $\Theta(n)$. Como *Merge Sort* realiza duas chamadas recursivas sobre metades da lista de tamanho N e, em seguida, uma chamada a *merge* sobre o resultado, o custo total do algoritmo é descrito pela recorrência:

$$T(N) = 2T(N/2) + \Theta(N)$$

em que o termo $2T(N/2)$ corresponde à ordenação recursiva das duas partições (Partição Esquerda e Partição Direita, conforme definidas na demonstração de corretude da seção 2.6.6) e o termo $\Theta(N)$ corresponde ao custo da intercalação final feita por *merge*.

Para resolver recorrências desse formato, recorre-se ao Teorema Mestre, que se aplica a qualquer recorrência da forma:

$$T(N) = a \cdot T(N/b) + f(N)$$

Nessa expressão, $a \geq 1$ representa a quantidade de subproblemas gerados em cada chamada recursiva, $b > 1$ indica o fator pelo qual o tamanho da entrada é reduzido, e $f(N)$ corresponde ao custo do trabalho realizado fora das chamadas recursivas. O teorema compara o crescimento de $f(N)$ com o de $N^{\log_b a}$, permitindo identificar a ordem assintótica da recorrência. De forma geral, quando $f(N)$ cresce mais lentamente, obtém-se $\Theta(N^{\log_b a})$; quando ambos crescem na mesma ordem, obtém-se $\Theta(N^{\log_b a}) \cdot \log N$; e, quando $f(N)$ cresce mais rapidamente, o resultado é $\Theta(f(N))$ (UC Davis, [s.d.]; NTU Ada, [s.d.]; Cormen et al., 2022).

Na recorrência do *Merge Sort*, cada chamada gera dois subproblemas, portanto $a = 2$. Cada subproblema possui metade do tamanho da entrada, de modo que $b = 2$. Além disso, o trabalho realizado fora das chamadas recursivas corresponde principalmente à intercalação das sublistas, cujo custo é $f(N) = \Theta(N)$. Por esse motivo, o Teorema Mestre pode ser aplicado diretamente. Calculando o termo de comparação, tem-se:

$$N^{\log_b a} = N^{\log_2 2} = N^1 = N$$

Como $f(N) = \Theta(N)$ apresenta a mesma ordem de crescimento de $N^{\log_b a}$, a recorrência se enquadra no segundo caso do Teorema Mestre. Portanto, a complexidade temporal do Merge Sort é:

$$T(N) = \Theta(N \log N)$$

Diferentemente da busca linear e da busca indexada, o Merge Sort não possui critério de parada antecipada nem depende da disposição inicial dos dados: a divisão ao meio e a intercalação completa ocorrem sempre, mesmo quando a entrada já está ordenada. Por isso, melhor caso, caso médio e pior caso coincidem em $\Theta(N \log N)$, o que contrasta com o sort nativo do Python (Timsort), que se aproxima de $\Theta(N)$ no melhor caso ao aproveitar sequências já ordenadas (Sedgewick; Wayne, 2011).

Em relação ao espaço, a intercalação exige um vetor auxiliar para armazenar os elementos combinados antes de sobrescrever a lista original. Como esse vetor tem tamanho proporcional a N em cada nível da recursão, mas os níveis não coexistem simultaneamente em memória, o espaço auxiliar total é $O(N)$ (Cormen et al., 2022; Morin, [s.d.]).

Aplicado à Configuração 3, o custo total combina a pontuação lexical, já analisada em 2.6.1, com a ordenação própria:

$$T_{merge} = \Theta(mL) + \Theta(N \log N)$$

que tem a mesma ordem assintótica dominante da busca linear com sort nativo ($\Theta(mL + N \log N)$), diferindo apenas em constantes de implementação, e não em complexidade. A vantagem do Merge Sort implementado não está em reduzir a ordem de crescimento, mas em oferecer uma garantia de pior caso previsível e verificável por prova formal (Cormen et al., 2022).

2.7.4 Síntese comparativa e fatores fora do modelo RAM

Quadro 5 - Síntese da complexidade das configurações avaliadas.

Algoritmo	Melhor caso	Caso médio/pior	Espaço
Busca linear	$\Theta(mL + N)$	$\Theta(mL + N \log N)$	$O(N)$
Busca indexada	$O(m)$	$O(m + P_q + mLr + r \log r)$ aproximando $\Theta(mL + N \log N)$ no pior caso	$O(r)$ em consultas + $O(L)$ de índice
Merge Sort (sobre a pontuação)	$\Theta(mL + N \log N)$	$\Theta(mL + N \log N)$	$O(N)$ auxiliar

Fonte: Elaborado pelos autores.

O modelo RAM assume operações elementares de custo constante e ignora fatores que afetam o desempenho real da implementação (Erickson, 2019). Entre eles: o custo de entrada/saída na leitura do corpus em CSV; efeitos de cache, já que a busca linear percorre os documentos sequencialmente (acesso com boa localidade), enquanto o índice invertido depende de acessos a estruturas de dicionário/hash, com localidade menos previsível; o overhead do interpretador Python, decorrente de tipagem dinâmica e do custo de chamadas como *list.count*; e a diferença entre o sort nativo do Python, implementado em C (Timsort), e a implementação própria do Merge Sort em Python puro, que tende a ser mais lenta na prática apesar de mesma ordem assintótica (Sedgewick; Wayne, 2011; Morin, [s.d.]).

2.8 Precision@5

A Precision@5 (P@5) foi utilizada para avaliar a qualidade dos resultados recuperados, representando a proporção de documentos relevantes entre os cinco primeiros resultados retornados para uma consulta. A métrica é definida por:

$$P@5(q) = |R_5(q) \cap Rel(q)| / 5 \quad (1)$$

Em que:

$R_5(q)$ = cinco primeiros documentos recuperados;

$Rel(q)$ = documentos relevantes;

Dessa forma, valores mais próximos de 1 indicam maior proporção de documentos relevantes entre os resultados apresentados no Top-5. A Precision@k é uma métrica amplamente utilizada na avaliação de sistemas de Recuperação de Informação (MANNING; RAGHAVAN; SCHÜTZE, 2008).

2.9 Desenho Experimental

Foi realizado um experimento controlado para comparar o desempenho de três estratégias de recuperação de informação: Busca Linear, Índice Invertido e Merge Sort. O experimento considerou dois tamanhos de corpus, com 500 e 1.000 artigos científicos, permitindo analisar o efeito do crescimento da coleção sobre o desempenho dos métodos. Para cada combinação entre tamanho do corpus, consulta e estratégia de recuperação, foram realizadas 30 execuções, sendo utilizadas as repetições para estimar de forma mais estável as

métricas de desempenho computacional. Foram calculadas as médias do tempo de consulta e do número de comparações, além do pico máximo de memória observado durante a execução.

A qualidade da recuperação foi avaliada por meio da Precision@5, utilizando um gabarito semântico comum às três estratégias. Dessa forma, a comparação entre os métodos foi realizada sob as mesmas consultas, documentos e critérios de relevância. Para o Índice Invertido, o custo de construção da estrutura de indexação foi medido separadamente, considerando o tempo de construção e o consumo de memória. Essa separação permite distinguir o custo inicial de preparação do índice do custo efetivo das consultas.

Os resultados foram posteriormente agregados por tamanho do corpus e estratégia de recuperação, utilizando estatísticas descritivas, como média e máximo, para analisar o comportamento das métricas. A comparação buscou identificar diferenças no desempenho dos métodos e verificar a existência de um possível trade-off entre tempo de execução, número de comparações, consumo de memória e qualidade da recuperação.

3. RESULTADOS

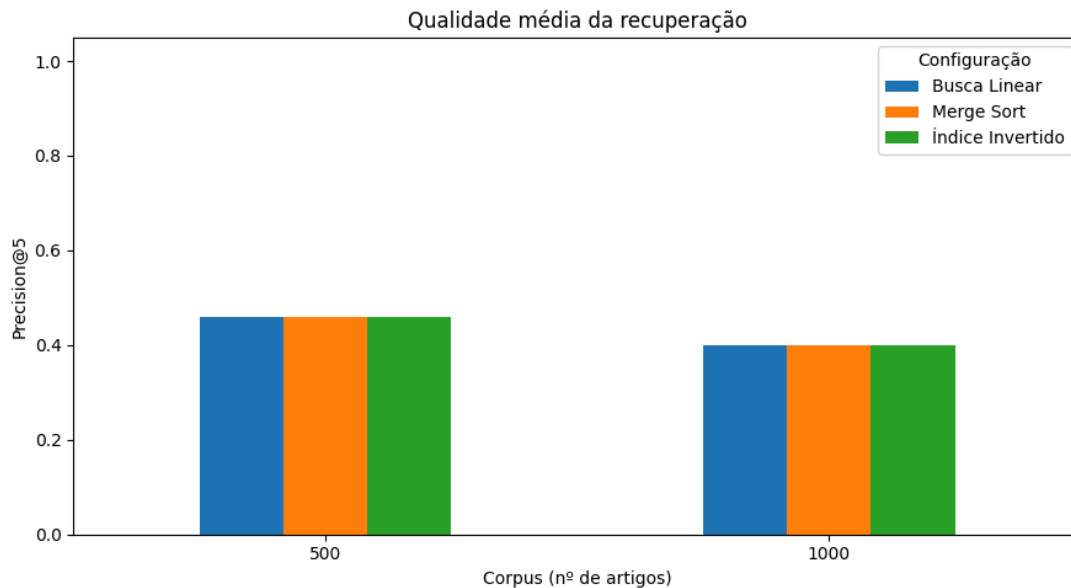
Os resultados obtidos na comparação entre Busca Linear, Índice Invertido e Merge Sort são apresentados na Tabela 1. Foram avaliados dois tamanhos de corpus, contendo 500 e 1.000 artigos, considerando como métricas o tempo médio de consulta, o número médio de comparações, a Precision@5 e o consumo de memória.

Observa-se inicialmente que o Índice Invertido apresentou os menores tempos médios de consulta para ambos os tamanhos de corpus. Para o corpus com 500 artigos, o método apresentou tempo médio de 0,5127 segundos, enquanto a Busca Linear e o Merge Sort apresentaram, ambos, 0,6271 segundos. Para o corpus de 1.000 artigos, o Índice Invertido manteve a vantagem, com tempo médio de 1,0207 segundos, em comparação com 1,2721 segundos para a Busca Linear e 1,2683 segundos para o Merge Sort. Esses resultados indicam que a utilização de uma estrutura de indexação foi capaz de reduzir o custo da recuperação à medida que o tamanho do corpus aumentou.

A análise do número de comparações reforça esse comportamento. No corpus de 500 artigos, a Busca Linear e o Merge Sort realizaram, em média, 500 comparações, enquanto o Índice Invertido apresentou média de 403,5 comparações. Para o corpus de 1.000 artigos, esses valores foram de 1.000 comparações para a Busca Linear e o Merge Sort e 779,4 para o Índice Invertido. Assim, o método baseado em índice apresentou uma redução aproximada de 19,3% e 22,1% no número de comparações para os corpora de 500 e 1.000 artigos, respectivamente.

Em relação à qualidade dos resultados recuperados, os três métodos apresentaram valores idênticos de Precision@5 para cada tamanho de corpus. Para o corpus de 500 artigos, a métrica apresentou valor médio de 0,46 para todas as configurações, enquanto para o corpus de 1.000 artigos o valor foi de 0,40. Esse comportamento indica que, nas condições avaliadas, as diferentes estratégias de recuperação não produziram diferenças na qualidade dos cinco primeiros resultados recuperados. Dessa forma, as diferenças observadas entre os métodos estão predominantemente associadas à eficiência computacional, e não à qualidade da recuperação, como ilustra a Figura 3.

Figura 3 - Precision@5 médio por configuração e tamanho de corpus.



Fonte: Elaborado pelos autores.

Como pode ser observado na figura, as barras de Busca Linear, Merge Sort e Índice Invertido praticamente se sobrepõem em cada tamanho de corpus, evidenciando que a escolha do algoritmo de busca ou ordenação não altera a qualidade dos resultados recuperados, apenas seu custo computacional.

Tabela 1 - Análise de Precisão e Custo computacional dos métodos de Busca

Corpus	Método	Tempo pré-proc. (s)	Tempo consulta (s)	Tempo total (s)	Comparações	Precision @5	Taxa de resultado vazio	Taxa sem item relevante	Memória média (MB)	Pico memória (MB)
500	Busca Linear	1,5582	0,6271	2,1853	500,0	0,46	0,00%	10,00%	0,0110	0,0123
500	Merge Sort	1,5582	0,6271	2,1853	500,0	0,46	0,00%	10,00%	0,0126	0,0135
500	Índice Invertido	1,5582	0,5127	2,0709	403,5	0,46	0,00%	10,00%	0,0388	0,0438
1000	Busca Linear	3,1848	1,2721	4,4567	1000,0	0,40	0,00%	10,00%	0,0221	0,0243
1000	Merge Sort	3,1848	1,2683	4,4529	1000,0	0,40	0,00%	10,00%	0,0257	0,0268
1000	Índice Invertido	3,1848	1,0207	4,2053	779,4	0,40	0,00%	10,00%	0,0488	0,0557

Fonte: Elaborado pelos autores.

Por outro lado, o Índice Invertido apresentou maior consumo de memória com apresentado na Tabela 1. Para o corpus de 500 artigos, o pico máximo de memória foi de 0,0438 MB, superior aos valores observados para a Busca Linear (0,0123 MB) e o Merge Sort (0,0135 MB). Para 1.000 artigos, o Índice Invertido apresentou pico máximo de 0,0557 MB, enquanto a Busca Linear e o Merge Sort apresentaram 0,0243 MB e 0,0268 MB, respectivamente.

Esse consumo adicional decorre do processamento necessário para resolver os candidatos a cada consulta, como a criação do conjunto de candidatos e o acesso às listas de

postagem dos termos consultados, e não do armazenamento do índice em si, cujo custo é medido separadamente e apresentado na Tabela 2.

Tabela 2 - Custo de construção do índice invertido e amortização por consulta.

Corpus	Tempo de construção do índice (s)	Pico de memória na construção (MB)	Tempo amortizado por consulta (s)
500	18,4537	1,0914	0,0615
1000	22,6288	1,8928	0,0754

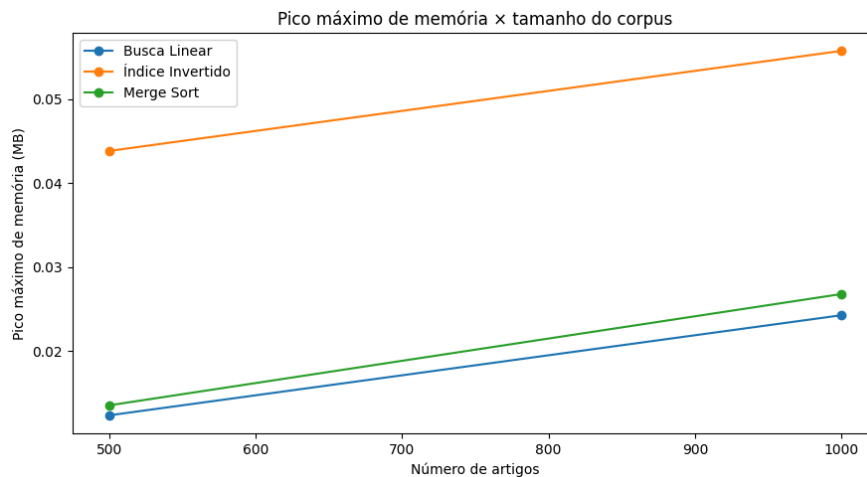
Fonte: Elaborado pelos autores.

A Tabela 2 detalha o custo de construção do índice invertido, medido separadamente das consultas. Para o corpus de 500 artigos, a construção levou 18, 4537 segundos e apresentou pico de memória de 1,0914 MB, valor muito superior ao consumo de memória observado durante as consultas individuais (0,0438 MB), o que confirma que o principal custo de espaço do índice está concentrado na sua construção, não no seu uso.

Para 1.000 artigos, o tempo de construção subiu para 22,6288 segundos e o pico de memória para 1,8928 MB. Quando esse custo é amortizado pelas 300 consultas realizadas por corpus, o acréscimo por consulta cai para 0,0615 segundos (500 artigos) e 0,0754 segundos (1.000 artigos), o equivalente a cerca de 12% e 7,4% do tempo médio de consulta do Índice Invertido nesses cenários.

Mesmo somando esse custo amortizado ao tempo de consulta, o Índice Invertido ainda apresenta tempo inferior ao da Busca Linear: $0,5127 + 0,0615 = 0,5742$ s, contra 0,6271 s, no corpus de 500 artigos; e $1,0207 + 0,0754 = 1,0961$ s, contra 1,2721 s, no corpus de 1.000 artigos, reforçando que a vantagem da indexação se mantém mesmo considerando o custo inicial de sua construção.

Figura 4 - Distribuição do Pico de Memória pelos métodos de busca



Fonte: Elaborado pelos autores.

De maneira geral, os resultados evidenciam um trade-off entre tempo de execução e consumo de memória. O Índice Invertido apresentou vantagem consistente em eficiência, reduzindo tanto o tempo de consulta quanto o número de comparações, ao custo de maior utilização de memória. A Busca Linear, por sua vez, apresentou o menor consumo de memória, mas demandou maior número de comparações e maior tempo de consulta. O Merge Sort apresentou comportamento semelhante à Busca Linear nas condições avaliadas, não proporcionando redução significativa no número de comparações ou no tempo de consulta.

Assim, considerando conjuntamente as métricas analisadas, o Índice Invertido mostrou-se a alternativa mais eficiente para os cenários avaliados, especialmente quando o objetivo é reduzir o custo de consulta em corporações maiores. Entretanto, sua adoção envolve um custo adicional de memória, evidenciando a necessidade de considerar as restrições de recursos computacionais na escolha da estratégia de recuperação.

4 DISCUSSÃO

Nesta seção são discutidas a escalabilidade das três configurações avaliadas, sua relação com a recuperação de contexto em aplicações de inteligência artificial generativa e as limitações do experimento realizado.

4.1 Escalabilidade, custos e trade-offs

A comparação entre as três configurações evidencia um trade-off entre o custo de preparação e o custo de consulta. A busca linear não exige a construção de uma estrutura auxiliar, mas seu custo de consulta cresce linearmente em função de L , pois o conteúdo do corpus precisa ser percorrido a cada execução. O índice invertido, por sua vez, exige um custo inicial de construção e espaço adicional $O(L)$, mas reduz o trabalho realizado nas consultas subsequentes ao restringir a pontuação aos r documentos candidatos presentes nas listas de postagens dos termos consultados.

A construção do índice tende a compensar seu custo quando a mesma estrutura é utilizada em várias consultas. Embora o experimento tenha considerado corpus de 500 e 1000 artigos, a vantagem da indexação tende a se tornar mais evidente em conjunto de documentos maiores e submetidos a um volume elevado de consultas, situação na qual o custo inicial pode ser amortizado. Entretanto, para consultas pouco frequentes ou corpus pequenos, o tempo empregado na construção do índice pode não ser compensado pela redução obtida no tempo de busca.

O Merge Sort atua em uma etapa diferente das buscas linear e indexada, pois sua finalidade é ordenar os documentos de acordo com as pontuações calculadas durante a recuperação. Sua complexidade temporal é $\Theta(N \log N)$ nos casos melhor, médio e pior, enquanto seu consumo adicional de memória é $\Theta(N)$. Em comparação com o método de ordenação nativo empregado nas outras configurações, o *Merge Sort* não reduz a ordem assintótica da etapa de ordenação, mas apresenta comportamento previsível mesmo no pior caso. Assim, sua inclusão no experimento permite analisar separadamente o custo da ordenação e comparar uma implementação explícita de algoritmo clássico com uma rotina otimizada da linguagem.

À medida que N e L crescem, a busca linear tende a se tornar menos viável, pois percorre repetidamente o conteúdo do corpus a cada consulta. O índice invertido pode apresentar melhor escalabilidade nas consultas, embora seu armazenamento também cresça

de acordo com o número de termos distintos e com o tamanho das listas de postagens, mantendo complexidade espacial $O(L)$.

Para conjunto de documentos muito maiores ou para aplicações que exigem recuperação semântica, a literatura apresenta estruturas de busca vetorial, como o *Faiss*, desenvolvidas para indexar e pesquisar embeddings densos de maneira eficiente, inclusive em conjuntos de dados que excedem a capacidade da memória RAM (Faiss, [s.d.]).

Frameworks como o *Sentence Transformers*, por sua vez, podem ser utilizados para gerar as representações vetoriais dos textos. Essas abordagens não substituem a análise dos algoritmos clássicos de busca e ordenação, mas representam uma direção complementar de escalabilidade. Em contrapartida, introduzem maior complexidade de infraestrutura e custos adicionais para gerar, armazenar, indexar e consultar os vetores densos (Gan et al., 2025).

4.2 Relação com a recuperação de contexto em aplicações de IA generativa (RAG)

Os k documentos retornados pelas configurações de recuperação poderiam compor o contexto fornecido a um sistema RAG, sendo organizados, por exemplo, em ordem decrescente de relevância antes de sua inserção no prompt. Nesse tipo de sistema, o modelo de linguagem utiliza os conteúdos recuperados como informações adicionais para produzir a resposta (Lewis et al., 2020).

O valor de k , o tamanho dos chunks e a ordem de apresentação dos conteúdos podem afetar diretamente o custo e a qualidade da geração. O uso de chunks maiores ou em maior quantidade amplia a cobertura de informações potencialmente relevantes, mas também aumenta o volume de tokens processados e pode diluir as informações úteis em meio a conteúdos irrelevantes. Além disso, a posição ocupada por cada trecho no prompt pode influenciar o quanto ele será efetivamente considerado pelo modelo durante a geração da resposta (Merola; Singh, 2025; Merola; Singh, 2025b; Bennani; Moslonka, 2026; Taiwo; Yusoff, 2026).

Um ponto central é que os algoritmos implementados realizam a recuperação com base em relevância lexical, representada pela frequência dos termos da consulta nos documentos, e não em relevância semântica. O gabarito construído com o modelo *all-MiniLM-L6-v2*, mencionado na Seção 2.5, evidencia essa diferença. Dois textos podem apresentar significados relacionados mesmo sem compartilhar exatamente os mesmos termos, relação que uma pontuação exclusivamente lexical pode não capturar.

Essa limitação é reconhecida na literatura sobre RAG (Survey on Knowledge-Oriented RAG, 2025) e pode favorecer documentos que utilizam vocabulário idêntico ao da consulta, mesmo quando outros documentos semanticamente relacionados apresentam terminologia diferente. Além disso, a qualidade da recuperação depende da atualização do corpus: se novos documentos não forem incorporados, o sistema poderá recuperar informações desatualizadas, ainda que o algoritmo de busca funcione corretamente.

As limitações da recuperação também se conectam a um risco relevante para aplicações reais. Mesmo quando um contexto é fornecido, o modelo de geração pode produzir afirmações não sustentadas pelos trechos recuperados ou atribuir peso desproporcional a informações pouco relevantes presentes no contexto (LeVine; Varjavand, 2025). Abordagens mais recentes procuram reduzir esse problema selecionando dinamicamente quais trechos devem ser efetivamente utilizados durante a geração (Sun et al., 2025) ou avaliando de forma explícita a qualidade do contexto recuperado em tarefas de RAG (Fukada; Keyaki; Matsui, 2025).

Nesse sentido, uma possível extensão do protótipo seria apresentar citações ou referências explícitas aos documentos de origem junto à resposta gerada. Esse recurso funcionaria como mecanismo de rastreabilidade, permitindo verificar se as afirmações produzidas pelo modelo estão efetivamente sustentadas pelos documentos presentes no top- k recuperado. Entretanto, a presença de citações não garante, isoladamente, a correção da resposta, pois ainda seria necessário verificar se cada afirmação está devidamente apoiada pelo conteúdo citado.

Por fim, existe um trade-off entre a precisão da recuperação, o tempo de resposta, o consumo de memória e o custo de geração. O aumento de k ou do tamanho dos chunks pode melhorar a cobertura de informações relevantes, mas também eleva a quantidade de tokens processados pelo modelo de linguagem e, consequentemente, o custo de inferência. Esse custo cresce com o tamanho do contexto fornecido ao modelo (Zhou et al., 2024; Jiang et al., 2024; Liu; Zaharia; Abbeel, 2024). Trata-se do mesmo equilíbrio entre velocidade, armazenamento e qualidade discutido por Gan et al. (2025) ao abordar a eficiência externa de sistemas RAG.

4.3 Limitações e ameaças à validade do experimento

Em relação à validade interna, o experimento contemplou 10 consultas e 30 repetições para cada combinação entre tamanho do corpus e configuração avaliada, totalizando 1.800

execuções mensuráveis. Embora as repetições permitam analisar o comportamento médio do tempo de execução, o conjunto de 10 consultas ainda pode não representar toda a diversidade de tamanhos, vocabulários e níveis de dificuldade encontrados em situações reais de recuperação de informação.

Em relação à validade de construto, os algoritmos utilizam uma pontuação lexical baseada na frequência dos termos, enquanto o gabarito empregado no cálculo da *Precision@5* foi construído por similaridade semântica com o modelo de embeddings *all-MiniLM-L6-v2*. Essa diferença permite avaliar em que medida os resultados lexicais coincidem com uma referência semântica, mas também introduz uma limitação: o gabarito é produzido por um único modelo e constitui uma aproximação automática de relevância, não um padrão-ouro elaborado e validado por especialistas humanos. Portanto, os valores de *Precision@5* devem ser interpretados como indicadores experimentais, e não como medidas definitivas da qualidade da recuperação.

Em relação à validade externa, o corpus está restrito a títulos e resumos em inglês provenientes da ACL Anthology. Essa delimitação reduz a possibilidade de generalizar os resultados para conjunto de documentos pertencentes a outros domínios, escritos em outros idiomas ou compostos pelo texto integral dos documentos. Também não é possível concluir, com base nos tamanhos testados, que o mesmo comportamento prático será mantido em conjuntos com milhares ou milhões de documentos, embora a análise assintótica permita estimar as tendências de crescimento.

Há ainda ameaças relacionadas à reprodutibilidade. Fatores como versão do Python, versões das bibliotecas, hardware, sistema operacional, processos executados simultaneamente e mecanismos internos de gerenciamento de memória podem afetar os tempos medidos. Esses metadados precisam ser documentados para permitir a reprodução do experimento e a comparação dos resultados entre execuções realizadas em ambientes diferentes.

Por fim, deve-se considerar uma ameaça relacionada ao uso de IA generativa como apoio. A literatura recente indica que modelos de linguagem podem gerar testes ou códigos aparentemente plausíveis sem cobrir adequadamente os caminhos de execução relevantes (Wang et al., 2025). Além disso, configurações consideradas determinísticas ainda podem apresentar variações entre execuções, o que exige controle e documentação do ambiente utilizado ao relatar resultados obtidos com apoio de IA (Atil et al., 2024).

5 DECLARAÇÃO DE USO DE IA GENERATIVA

Em conformidade com o Código de Conduta da Sociedade Brasileira de Computação (SBC) para pessoas autoras de publicações, os autores declaram que ferramentas de Inteligência Artificial (IA) Generativa foram utilizadas no desenvolvimento deste trabalho, nos seguintes termos:

Foi utilizada a ferramenta ChatGPT, desenvolvida pela OpenAI, como apoio nas seguintes etapas:

Revisão do protótipo e do texto: utilizada na revisão do código-fonte do protótipo implementado (Busca Linear, Índice Invertido e Merge Sort), auxiliando na identificação de possíveis inconsistências, na sugestão de melhorias pontuais e na revisão de trechos de implementação.

Revisão textual do relatório: apoio na revisão da redação, na clareza da linguagem técnica e na verificação da consistência entre as seções do relatório, incluindo a formalização de trechos da metodologia e da discussão.

Geração de elementos visuais e gráficos: apoio na criação e formatação dos gráficos apresentados nas Figuras 3 e 4 (Precision@5 médio e distribuição do pico de memória), bem como de elementos visuais de apoio à apresentação dos resultados.

Todo o conteúdo gerado ou revisado com o apoio da ferramenta foi analisado, validado e, quando necessário, ajustado pelos autores, que permanecem integralmente responsáveis pela exatidão, originalidade e integridade de todo o conteúdo do trabalho, incluindo a ausência de plágio. A ferramenta de IA Generativa não é listada como autora deste artigo, e sua utilização não substituiu o julgamento científico, a análise metodológica ou a interpretação dos resultados, que foram conduzidos pelos autores.

6 TABELA DE CONTRIBUIÇÃO INDIVIDUAL

Quadro 6 - Contribuição individual dos integrantes.

Integrante	Responsabilidade principal	Contribuições específicas
Clariane Maria Oliveira Lisboa	Definição formal do problema, relevância lexical e Busca Linear	Definiu entradas, saída, pré e pós-condições do problema; implementou a pontuação lexical (score_documento) e a busca_linear; definiu as consultas utilizadas na avaliação.
Iris Azenate Ferreira	Coleta, limpeza e chunking do corpus	Coletou os metadados dos artigos via pacote acl-anthology, selecionou os candidatos relacionados ao tema, organizou o corpus final em CSV e definiu a estratégia de chunking (1 artigo = 1 chunk).
José Nilton dos Santos Oliveira	Índice invertido e busca indexada	Implementou a construção do índice invertido (construir_indice_invertido) e a busca indexada (busca_indexada). Responsável por selecionar e pontuar apenas os documentos candidatos identificados pelo índice.
Leonardo Souza Silva	Merge Sort e corretude	Implementou merge_sort e merge e a configuração busca_merge_sort; conduziu a demonstração de corretude por indução forte e invariante de laço.
Thiago de Jesus dos Santos	Experimentos, resultados e reprodutibilidade	Executou as medições do experimento (1800 execuções), registrou tempo, comparações, Precision@5 e memória, e gerou as tabelas-resumo e os gráficos de resultados. Realizou os testes de casos de borda das três configurações.
Victória Cardoso dos Santos	Análise assintótica, RAG e limitações	Realizou a análise no modelo RAM dos três algoritmos (melhor, pior e caso médio), relacionou a recuperação ao contexto de RAG e redigiu a discussão de escalabilidade, trade-offs e limitações/ameaças à validade.

Fonte: Elaborado pelos autores.

Além das responsabilidades descritas, os integrantes também colaboraram na revisão geral do trabalho, na preparação da apresentação e na gravação do vídeo. A organização e a disponibilização dos arquivos no repositório ficaram sob responsabilidade de Leonardo Souza Silva.

7 URL DO VÍDEO E REPOSITÓRIO

Repositório Github:

https://github.com/Leonardodcomp/PAA_UFS_2026_2_Ferreira_Iris_Ferreira_Jose_Lisboa_Clariane_Santos_Thiago_Santos_Victoria_Silva_Leo.git

URL do vídeo:

<https://drive.google.com/file/d/1tM8qdJzN7ZpnDLzEfTQ7DrRIJ9Djf31t/view?usp=sharing>

REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIATION FOR COMPUTATIONAL LINGUISTICS. **ACL Anthology**. [S. l.]: Association for Computational Linguistics, [s. d.]. Disponível em: <https://aclanthology.org/>. Acesso em: 23 set. 2026.

ATIL et al. **Non-Determinism of "Deterministic" LLM Settings**. arXiv preprint arXiv:2408.04667, 2024. Disponível em: <https://arxiv.org/abs/2408.04667>.

BENNANI, Sofia; MOSLONKA, Charles. **A Systematic Analysis of Chunking Strategies for Reliable Question Answering**. arXiv preprint arXiv:2601.14123, 2026. Disponível em: <https://arxiv.org/abs/2601.14123>. Acesso em: 20 set. 2026.

CREATIVE COMMONS. **Attribution 4.0 International (CC BY 4.0)**. Disponível em: <https://creativecommons.org/licenses/by/4.0/>. Acesso em: 15 set. 2026.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms**. 4. ed. Cambridge, MA: MIT Press, 2022.

ERICKSON, Jeff. **Algorithms**. Urbana: [s.n.], 2019. Disponível em: <https://jeffe.cs.illinois.edu/teaching/algorithms/>. Acesso em: 15 set. 2026.

FAISS. **Faiss Documentation**. [S.l.: s.n., s.d.]. Disponível em: <https://faiss.ai/>. Acesso em: 20 set. 2026.

FUKADA; KEYAKI; MATSUI. UTokyo-HitU at TREC 2025 RAG Track: HyDE-Enhanced Sparse-Dense Retrieval Fusion with LLM Reranking. In: **TREC 2025 RAG Track**, 2025. Disponível em: <https://trec.nist.gov/pubs/trec34/papers/UTokyo.rag.pdf>.

GAN et al. **Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey**. arXiv preprint arXiv:2504.14891, 2025. Disponível em: <https://arxiv.org/abs/2504.14891>.

JIANG, H. et al. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. In: **Advances in Neural Information Processing Systems (NeurIPS 2024)**, 2024. Disponível em: https://proceedings.neurips.cc/paper_files/paper/2024/hash/5dfbe6f5671e82c76841ba687a8a9ecb-Abstract-Conference.html.

LEI; CHANG; LIPOVETZKY; EHINGER. Planning-Driven Programming: A Large Language Model Programming Workflow. In: **Annual Meeting of the Association for Computational Linguistics (ACL 2025)**, 2025. Disponível em: <https://aclanthology.org/2025.acl-long.621/>.

LEVINE; VARJAVAND. **Relevance Isn't All You Need: Scaling RAG Systems With Inference-Time Compute Via Multi-Criteria Reranking**. arXiv preprint arXiv:2504.07104, 2025. Disponível em: <https://arxiv.org/abs/2504.07104>.

LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: **Advances in Neural Information Processing Systems (NeurIPS 2020)**, 2020. Disponível em: <https://arxiv.org/abs/2005.11401>. Acesso em: 20 set. 2026.

LIU et al. **Enhancing the Robustness of LLM-Generated Code**. arXiv preprint, 2025. Disponível em: <https://arxiv.org/html/2503.20197v1>.

LIU; ZAHARIA; ABBEEL. RingAttention with Blockwise Transformers for Near-Infinite Context. In: **International Conference on Learning Representations (ICLR 2024)**, 2024. Disponível em: <https://openreview.net/forum?id=WsRHpHH4s0>.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge: Cambridge University Press, 2008.

MEROLA, Carlo; SINGH, Jaspinder. **Reconstructing Context: Evaluating Advanced Chunking Strategies for Retrieval-Augmented Generation**. arXiv preprint arXiv:2504.19754, 2025a. Disponível em: <https://arxiv.org/abs/2504.19754>. Acesso em: 20 set. 2026.

MEROLA, Carlo; SINGH, Jaspinder. The Chunking Paradigm: Recursive Semantic for RAG Optimization. In: **ICNLSP 2025**, 2025b. Disponível em: <https://aclanthology.org/2025.icnlp-1.15.pdf>. Acesso em: 20 set. 2026.

MORIN, Pat. **Open Data Structures**. [S.l.: s.n., s.d.]. Disponível em: <https://opendatastructures.org/>. Acesso em: 20 set. 2026.

NTU ADA. **Divide-and-Conquer**. [S.l.: s.n., s.d.]. Disponível em: https://www.csie.ntu.edu.tw/~yvchen/f106-ada/doc/171005_Divide-and-Conquer2.pdf. Acesso em: 20 set. 2026.

REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: **Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)**, 2019. Disponível em: <https://arxiv.org/abs/1908.10084>.

SEdgeWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. [S.l.: s.n.], 2011. Disponível em: <https://algs4.cs.princeton.edu/>. Acesso em: 20 set. 2026.

SENTENCE TRANSFORMERS. **all-MiniLM-L6-v2**. [S.l.]: Hugging Face, [2021?]. Disponível em: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Acesso em: 20 set. 2026.

Survey on Knowledge-Oriented Retrieval-Augmented Generation. arXiv preprint arXiv:2503.10677, 2025. Disponível em: <https://arxiv.org/abs/2503.10677>. Acesso em: 20 set. 2026.

SUN, J.; ZHONG, X.; ZHOU, S.; HAN, J. **DynamicRAG: Leveraging Outputs of Large Language Model as Feedback for Dynamic Reranking in Retrieval-Augmented Generation.** arXiv preprint arXiv:2505.07233, 2025. Disponível em: <https://arxiv.org/abs/2505.07233>.

TAIWO, Samuel; YUSOFF, Mohd Amaluddin. **Evaluating Chunking Strategies For Retrieval-Augmented Generation in Oil and Gas Enterprise Documents.** arXiv preprint arXiv:2603.24556, 2026. Disponível em: <https://arxiv.org/abs/2603.24556>. Acesso em: 20 set. 2026.

UC DAVIS ECS122A. **Divide-and-Conquer Recurrences and the Master Theorem.** [S.l.: s.n., s.d.]. Disponível em: <https://www.cs.ucdavis.edu/~bai/ECS122A/Notes/mastertheorem.pdf>. Acesso em: 20 set. 2026.

WANG et al. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. In: **Findings of the Association for Computational Linguistics: NAACL 2025**, 2025. Disponível em: <https://aclanthology.org/2025.findings-naacl.197/>.

ZHOU, Z. et al. A Survey on Efficient Inference for Large Language Models. arXiv preprint arXiv:2404.14294, 2024. Disponível em: <https://arxiv.org/abs/2404.14294>.

ZHOU, Z. et al. **A Survey on Efficient Inference for Large Language Models.** arXiv preprint arXiv:2404.14294, 2024. Disponível em: <https://arxiv.org/abs/2404.14294>. Acesso em: 20 set. 2026.